

Reputation Service MCP Server

Introduction

This guide details the **Reputation Service MCP Agent** workflow, designed to check the reputation of domains, URLs, IPs, and files using AI and the Model Context Protocol (MCP).

Overview

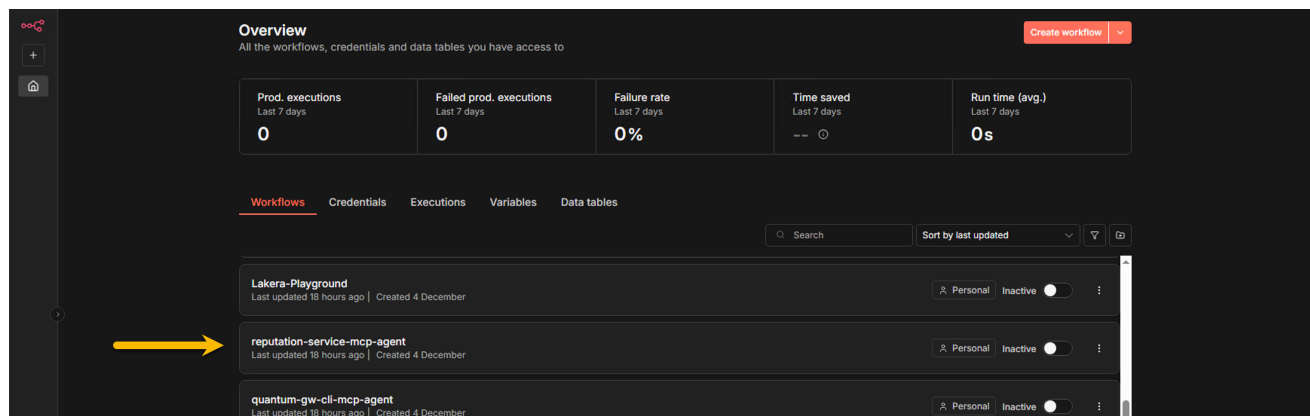
The **Reputation Service MCP Agent** allows users to quickly verify if a specific indicator (like a domain name or file hash) is malicious by querying Check Point's reputation services via natural language.

Prerequisites

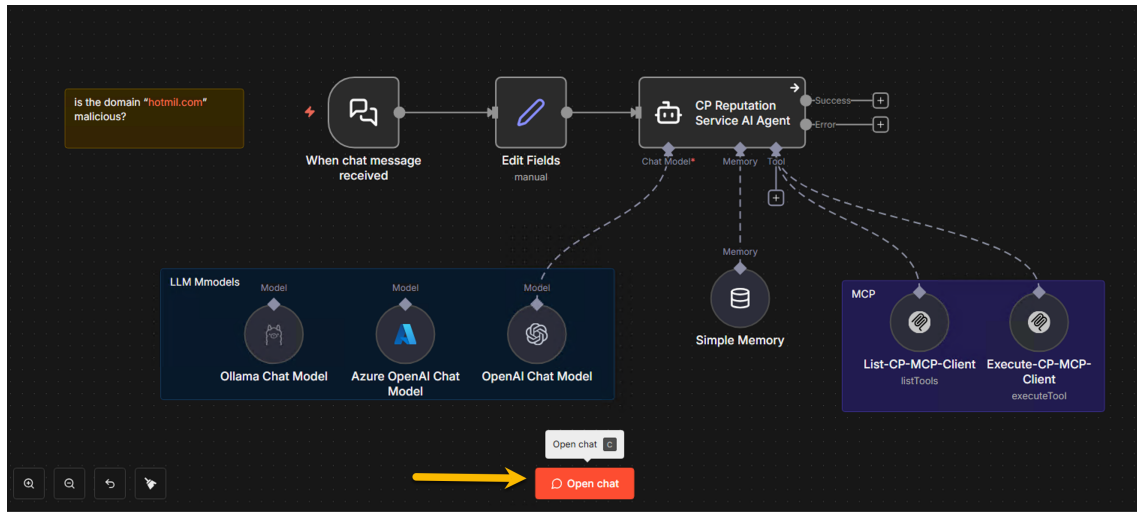
- **n8n URL:** <http://10.1.3.33:5678>
- **Credentials:**
 - **OpenAI / Azure OpenAI / Ollama:** For the LLM.
 - **MCP Client:** HTTP connection to the Reputation Service MCP server.

Task 1: Open the Workflow

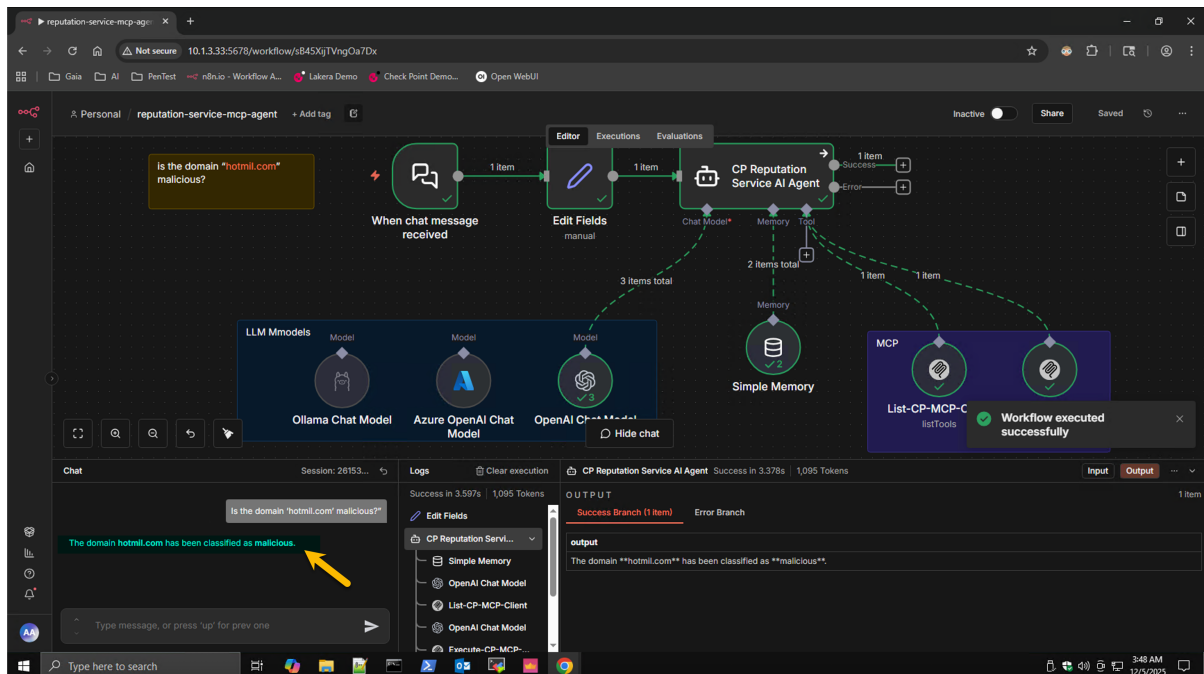
1. Log in to n8n with the email address admin@cpdemo.com and password: **Cpwins!1**
2. Locate and click on the **reputation-service-mcp-agent** workflow.



3. Locate and click the **Open Chat** button.



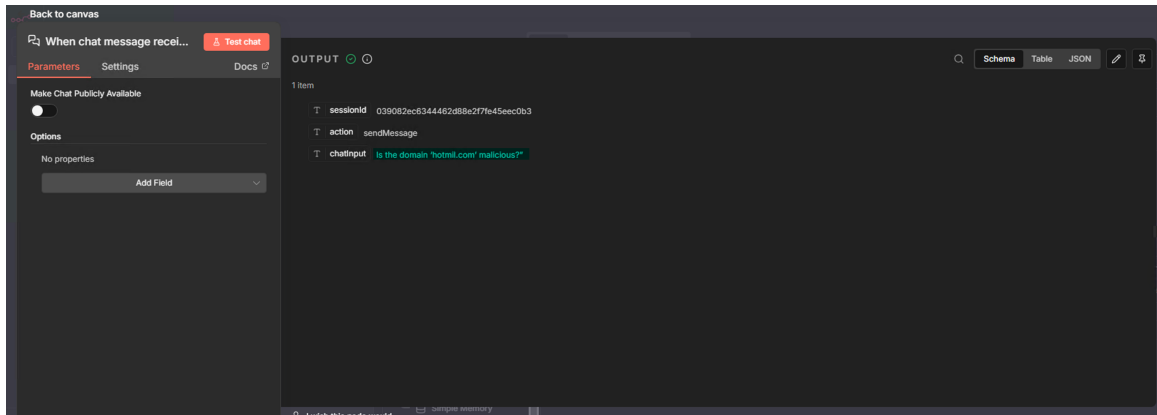
4. Enter a query, for example: > *"Is the domain 'hotmil.com' malicious?"*
5. The AI agent will query the reputation service via MCP and report the classification (e.g., Malicious, Benign) and other relevant details.



Task 2: Workflow Deep Dive

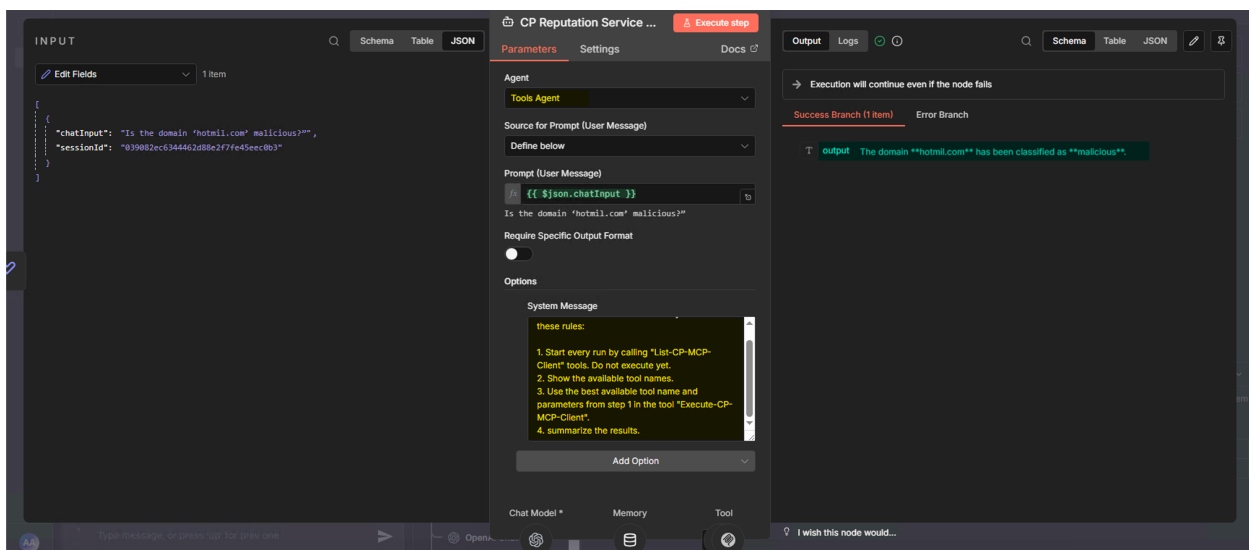
1. Input Trigger: Chat Interface

- **Node Name:** When chat message received
- **Purpose:** Captures the user's reputation query.



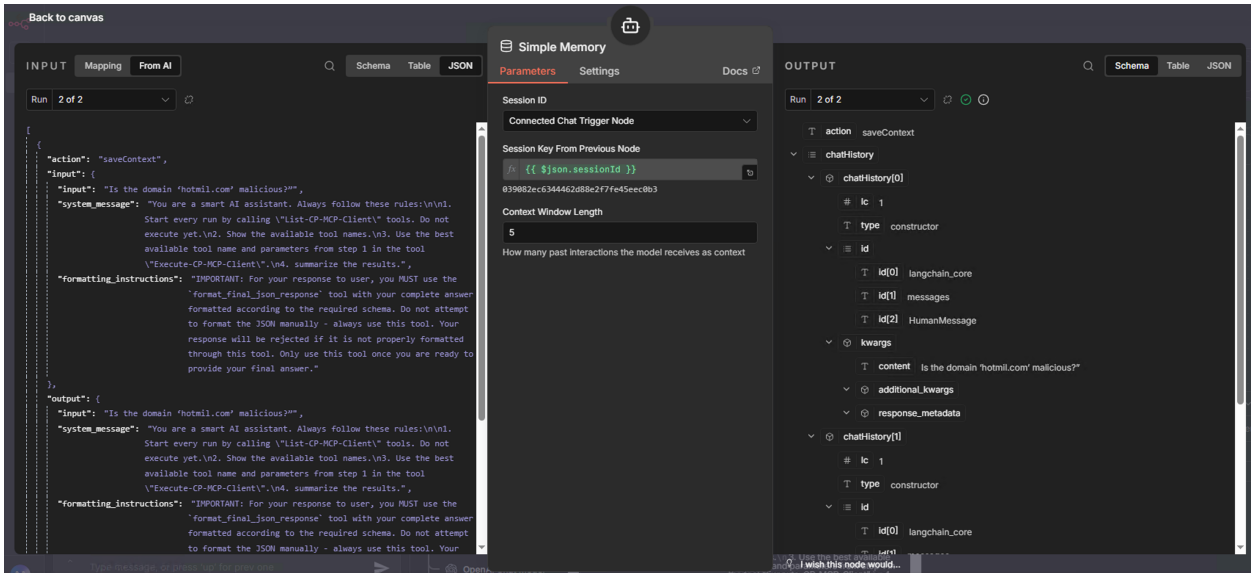
2. The Brain: AI Agent

- **Node Name:** CP Reputation Service AI Agent
- **Type:** @n8n/n8n-nodes-langchain.agent
- **Purpose:** Identifies the indicator in the user's message and calls the reputation tool.
- **Configuration:**
 - **System Message:** Instructs the AI to list tools and then execute the reputation check tool. System Messages are a crucial component in AI agents.



3. Memory Management

- **Node Name:** Simple Memory
- **Purpose:** Maintains conversation context.



The screenshot shows the configuration for the 'Simple Memory' node. The 'Parameters' tab is active, showing the following settings:

- Session ID:** Connected Chat Trigger Node
- Session Key From Previous Node:** `{{ $json.sessionId }}`
- Session Key:** `039082ec6344462d88e2f7fe45ec0b3`
- Context Window Length:** 5
- How many past interactions the model receives as context:** 5

The 'INPUT' tab shows a JSON object with the following structure:

```

{
  "action": "saveContext",
  "input": {
    "input": "Is the domain 'hotmail.com' malicious?",
    "system_message": "You are a smart AI assistant. Always follow these rules:\n1. Start every run by calling '\list-CP-MCP-Client' tools. Do not execute yet.\n2. Show the available tool names.\n3. Use the best available tool name and parameters from step 1 in the tool.\n4. Execute-CP-MCP-Client.\n5. summarize the results.",
    "formatting_instructions": "IMPORTANT: For your response to user, you MUST use the 'format_final_json_response' tool with your complete answer formatted according to the required schema. Do not attempt to format the JSON manually - always use this tool. Your response will be rejected if it is not properly formatted through this tool. Only use this tool once you are ready to provide your final answer."
  },
  "output": {
    "input": "Is the domain 'hotmail.com' malicious?",
    "system_message": "You are a smart AI assistant. Always follow these rules:\n1. Start every run by calling '\list-CP-MCP-Client' tools. Do not execute yet.\n2. Show the available tool names.\n3. Use the best available tool name and parameters from step 1 in the tool.\n4. Execute-CP-MCP-Client.\n5. summarize the results.",
    "formatting_instructions": "IMPORTANT: For your response to user, you MUST use the 'format_final_json_response' tool with your complete answer formatted according to the required schema. Do not attempt to format the JSON manually - always use this tool. Your response will be rejected if it is not properly formatted through this tool. Only use this tool once you are ready to provide your final answer."
  }
}

```

The 'OUTPUT' tab shows the resulting JSON object:

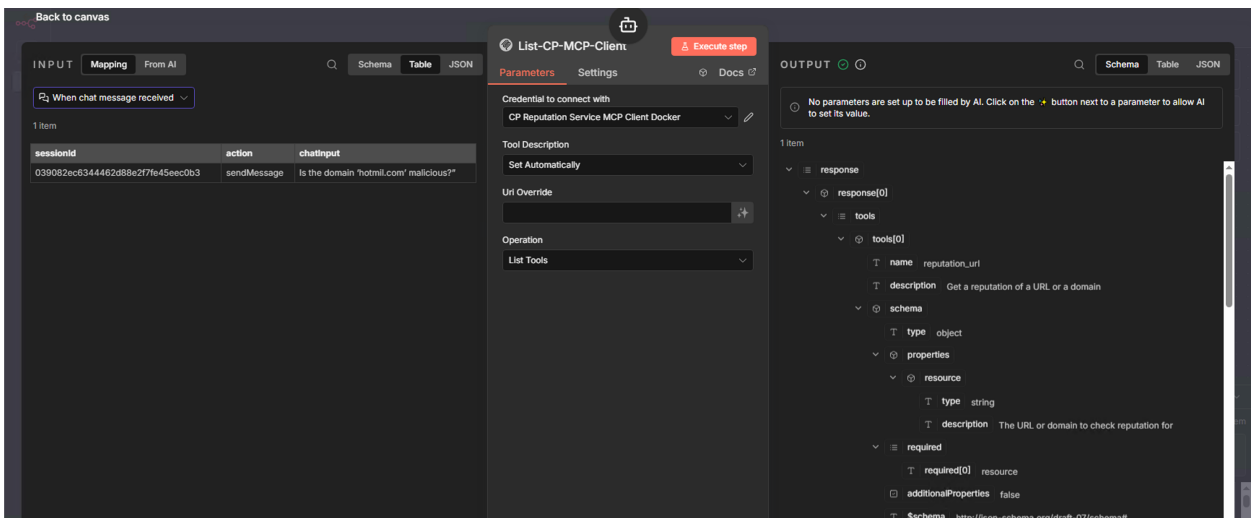
```

{
  "action": "saveContext",
  "chatHistory": [
    {
      "lc": 1,
      "type": "constructor",
      "id": [
        { "lc": 0, "type": "langchain_core", "messages" },
        { "lc": 1, "type": "messages" },
        { "lc": 2, "type": "HumanMessage" }
      ],
      "kwargs": {
        "content": "Is the domain 'hotmail.com' malicious?"
      },
      "additional_kwargs": {},
      "response_metadata": {}
    },
    {
      "lc": 1,
      "type": "constructor",
      "id": [
        { "lc": 0, "type": "langchain_core" }
      ]
    }
  ]
}

```

4. Tool: MCP Client

- **Node:** List-CP-MCP-Client.
- **Purpose:** Interfaces with the Reputation Service MCP server.
- **Functionality:**
 - **List:** Shows available reputation tools.



The screenshot shows the configuration for the 'List-CP-MCP-Client' node. The 'Parameters' tab is active, showing the following settings:

- Credential to connect with:** CP Reputation Service MCP Client Docker
- Tool Description:** Set Automatically
- Uri Override:** (empty)
- Operation:** List Tools

The 'INPUT' tab shows a table with the following data:

sessionId	action	chatInput
039082ec6344462d88e2f7fe45ec0b3	sendMessage	Is the domain 'hotmail.com' malicious?

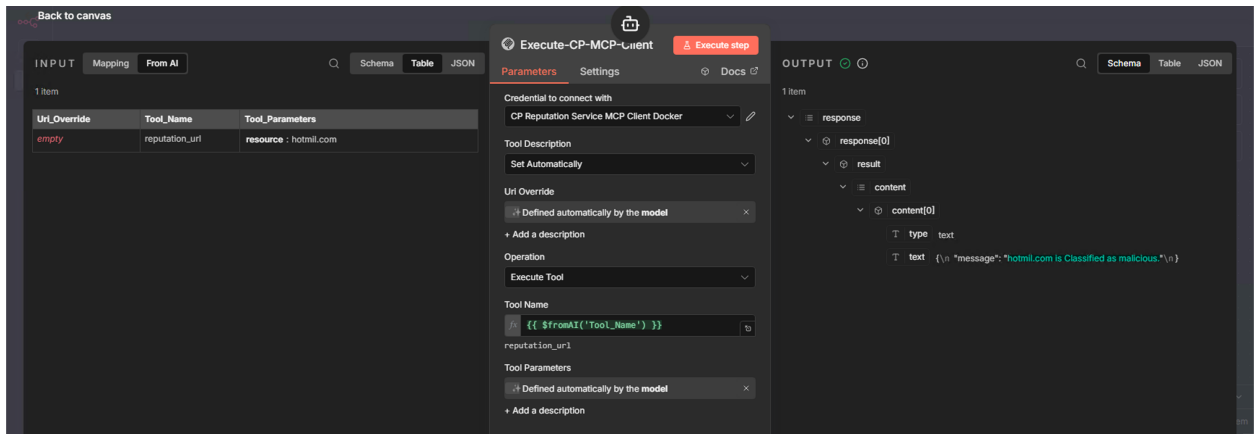
The 'OUTPUT' tab shows the resulting JSON object:

```

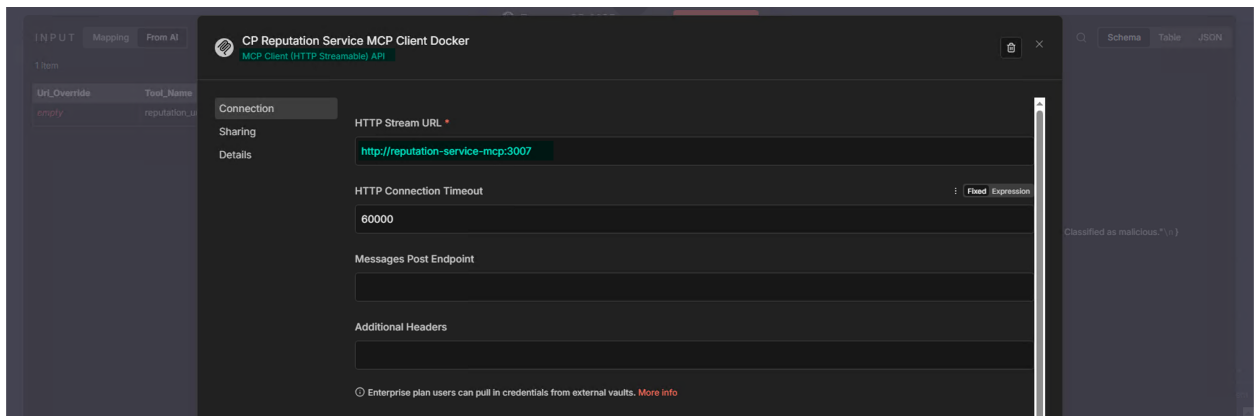
{
  "response": [
    {
      "tools": [
        {
          "name": "reputation_url",
          "description": "Get a reputation of a URL or a domain",
          "schema": {
            "type": "object",
            "properties": {
              "resource": {
                "type": "string",
                "description": "The URL or domain to check reputation for",
                "required": true
              }
            },
            "required": [
              "resource"
            ],
            "additionalProperties": false
          },
          "$schema": "http://json-schema.org/draft-07/schema#"
        }
      ]
    }
  ]
}

```

- **Node:** **Execute-CP-MCP-Client**
- **Functionality:**
 - **Execute:** Runs the check for the specific indicator.



Under “Credentials to connect with”, we point to the MCP server installed within a docker container (API_KEY). Read More [here](#).



Best Practices

- **Clear Indicators:** Ensure the domain, IP, or hash is clearly visible in your message.
- **Bulk Checks:** You can try asking about multiple indicators, though results depend on the specific tool's capabilities.